

VIT EMOTION DETECTOR

```
!pip install -q git+https://github.com/huggingface/transformers
datasets
```

```
Installing build dependencies ... ents to build wheel ...
etadata ... ers (PEP 517) ...
```

```
from datasets import *
from transformers import ViTFeatureExtractor
from transformers import ViTModel
from transformers import TrainingArguments, Trainer
from transformers.modeling_outputs import SequenceClassifierOutput
import numpy as np
import pandas as pd
import torch.nn as nn
import pickle
from matplotlib import pyplot as plt
%matplotlib inline
import seaborn as sns
from sklearn.metrics import confusion_matrix
```

Preprocessing

I finetune ViT on the Facial Expression Recognition 2013 (FER-2013) dataset, which consists of 35.887 48x48 pixel grayscale images representing 7 different emotions. It is preprocessed using ViTFeatureExtractor, which resizes every image to the resolution that the model expects, i.e. 224x224, and normalizes the channels.

Dataset: <https://www.kaggle.com/c/challenges-in-representation-learning-facial-expression-recognition-challenge>

Dataset Paper: <https://arxiv.org/pdf/1307.0414.pdf>

```
string_labels = ['Anger', 'Disgust', 'Fear', 'Happiness', 'Sadness',
                 'Surprise', 'Neutral']
```

```
feature_extractor = ViTFeatureExtractor.from_pretrained('google/vit-
base-patch16-224-in21k')
```

Load original data into a DataFrame

```
fer_df = pd.read_csv("fer2013.csv") # available on kaggle
```

```
fer_df.head()
```

	emotion																pixels
Usage																	
0	0	70	80	82	72	58	58	60	63	54	58	60	48	89	115	121...	

```

Training
1      0  151 150 147 155 148 133 111 140 170 174 182 15...
Training
2      2  231 212 156 164 174 138 161 173 182 200 106 38...
Training
3      4   24 32 36 30 32 23 19 20 30 41 21 22 32 34 21 1...
Training
4      6   4 0 0 0 0 0 0 0 0 0 0 0 3 15 23 28 48 50 58 84...
Training

```

First preprocessing step

```

def prepare_fer_data(data):
    """ Prepare fer data for vision transformer
        input: fer df loaded from csv
        output: df that can be load into a huggingface dataset """

    # outputs
    image_list = []
    image_labels = list(map(int, data['emotion']))

    # go over all images
    for i, row in enumerate(data.index):
        image = np.fromstring(data.loc[row, 'pixels'], dtype=int,
sep=' ')
        image = np.reshape(image, (48, 48))
        # adapt grayscale to rgb format (change single values to
triplets of the same value)
        image = image[..., np.newaxis]
        image = np.repeat(image, 3, axis=2)
        # convert to list format used by the later functions
        image = image.astype(int).tolist()
        # save to output
        image_list.append(image)

    output_df = pd.DataFrame(list(zip(image_list, image_labels)),
                             columns=['img', 'label'])

    return output_df

fer_train_df = prepare_fer_data(fer_df[fer_df['Usage']=='Training'])
fer_test_df = prepare_fer_data(fer_df[fer_df['Usage']=='PrivateTest'])
fer_val_df = prepare_fer_data(fer_df[fer_df['Usage']=='PublicTest'])

fer_train_df.head()

```

	img	label
0	[[[70, 70, 70], [80, 80, 80], [82, 82, 82], [7...	0
1	[[[151, 151, 151], [150, 150, 150], [147, 147,...	0
2	[[[231, 231, 231], [212, 212, 212], [156, 156,...	2

```

3  [[[24, 24, 24], [32, 32, 32], [36, 36, 36], [3...      4
4  [[[4, 4, 4], [0, 0, 0], [0, 0, 0], [0, 0, 0], ...      6

print(len(fer_train_df))
print(len(fer_test_df))
print(len(fer_val_df))

28709
3589
3589

train_ds = Dataset.from_pandas(fer_train_df)
#undersample train due to memory issues on google colab
splits = train_ds.train_test_split(test_size=0.15)
train_ds = splits["train"]

val_ds = Dataset.from_pandas(fer_val_df)
test_ds = Dataset.from_pandas(fer_test_df)

train_ds
Dataset({
  features: ['img', 'label'],
  num_rows: 24402
})

# image size
np.array(train_ds[0]["img"]).shape

(48, 48, 3)

```

Second preprocessing step using the ViT feature extractor

```

def preprocess_images(examples):
    """ Prepare datasets for vision transformer
    input: dataset with images in their original size
    output: dataset with pixelvalues computed by the feature extractor
    added """
    # get batch of images
    images = examples['img']
    # convert to list of NumPy arrays of shape (C, H, W)
    images = [np.array(image, dtype=np.uint8) for image in images]
    images = [np.moveaxis(image, source=-1, destination=0) for image
in images]
    # preprocess and add pixel_values
    inputs = feature_extractor(images=images)
    examples['pixel_values'] = inputs['pixel_values']

    return examples

```

```

# features of the new dataset with an additional column for the
preprocess 224x224x3 images
features = Features({
    'label': ClassLabel(names=['Angry', 'Disgust', 'Fear', 'Happy',
'Sad', 'Surprise', 'Neutral']),
    'img': Array3D(dtype="int64", shape=(3,48,48)),
    'pixel_values': Array3D(dtype="float32", shape=(3, 224, 224)),
})

preprocessed_train_ds = train_ds.map(preprocess_images, batched=True,
batch_size=1, features=features)
with open('preprocessed_train_ds.pickle', 'wb') as handle:
    pickle.dump(preprocessed_train_ds, handle,
protocol=pickle.HIGHEST_PROTOCOL)
#preprocessed_val_ds = val_ds.map(preprocess_images, batched=True,
features=features)
#with open('preprocessed_val_ds.pickle', 'wb') as handle:
#    pickle.dump(preprocessed_val_ds, handle,
protocol=pickle.HIGHEST_PROTOCOL)
#preprocessed_test_ds = test_ds.map(preprocess_images, batched=True,
features=features)
#with open('preprocessed_test_ds.pickle', 'wb') as handle:
#    pickle.dump(preprocessed_test_ds, handle,
protocol=pickle.HIGHEST_PROTOCOL)

preprocessed_train_ds

{"model_id": "90484c446ef74ebb925c0bbd9309882c", "version_major": 2, "vers
ion_minor": 0}

Dataset({
    features: ['img', 'label', 'pixel_values'],
    num_rows: 24402
})

# final image size
print(len(preprocessed_train_ds[0]["pixel_values"]))
print(len(preprocessed_train_ds[0]["pixel_values"][0]))
print(len(preprocessed_train_ds[0]["pixel_values"][0][0]))

3
224
224

```

Looking at the Data

Let's quickly look at some samples of processed data as well as the label distribution.

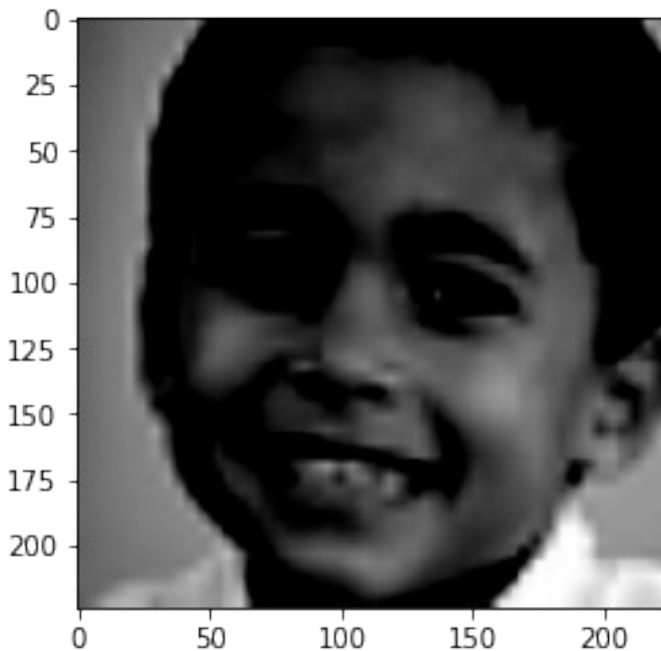
```

# show a single image from the dataset
data_for_image = np.array(preprocessed_train_ds[0]["pixel_values"])
data_for_image[data_for_image < 0] = 0

plt.imshow(np.transpose(data_for_image, (1,2,0)),
            interpolation='nearest')
plt.show()

print(string_labels[preprocessed_train_ds[0]["label"]])

```



Happiness

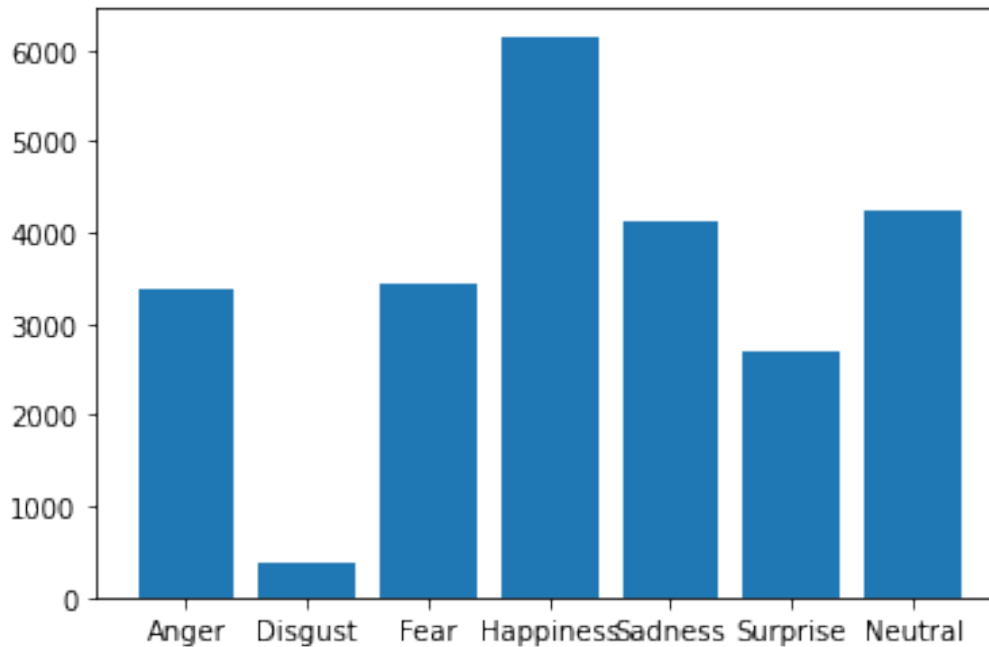
```

# show 100 images from the dataset
fig, axes = plt.subplots(10,10, figsize=(11,11))
for i,ax in enumerate(axes.flat):
    data_for_image = np.array(preprocessed_train_ds[i]["pixel_values"])
    data_for_image[data_for_image < 0] = 0
    ax.imshow(np.transpose(data_for_image, (1,2,0)),
              interpolation='nearest')
    ax.set_axis_off()

```



```
# distribution of labels in the training set
keys, counts = np.unique(preprocessed_train_ds["label"],
return_counts=True)
plt.bar(string_labels, counts)
plt.show()
```



Defining the model

The model architecture is defined in PyTorch, with dropout and a linear layer added on top of the ViT model's output of the special CLS token representing the input picture.

```
class ViTForImageClassification(nn.Module):
    #define architecture
    def __init__(self, num_labels=len(string_labels)):
        super(ViTForImageClassification, self).__init__()
        self.vit = ViTModel.from_pretrained('google/vit-base-patch16-224-in21k')
        self.dropout = nn.Dropout(0.1)
        self.classifier = nn.Linear(self.vit.config.hidden_size, num_labels)
        self.num_labels = num_labels

    #define a forward pass through that architecture + loss computation
    def forward(self, pixel_values, labels):
        outputs = self.vit(pixel_values=pixel_values)
        output = self.dropout(outputs.last_hidden_state[:,0])
        logits = self.classifier(output)

        loss = None
        if labels is not None:
            loss_fct = nn.CrossEntropyLoss()
            loss = loss_fct(logits.view(-1, self.num_labels), labels.view(-1))
```

```

    return SequenceClassifierOutput(
        loss=loss,
        logits=logits,
        hidden_states=outputs.hidden_states,
        attentions=outputs.attentions,
    )

```

Training uses the standard HuggingFace [Trainer](#) interface.

```

metric_name = "accuracy"

args = TrainingArguments(
    f"test-cifar-10",
    evaluation_strategy = "epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=6,
    weight_decay=0.01,
    load_best_model_at_end=True,
    metric_for_best_model=metric_name,
    logging_dir='logs',
)

metric = load_metric("accuracy")

def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1)
    return metric.compute(predictions=predictions, references=labels)

{"model_id": "8e7835f8facf4af4b1db6df177d34262", "version_major": 2, "version_minor": 0}

model = ViTForImageClassification()

{"model_id": "121e903627704fe68e76aca8fb9ebe29", "version_major": 2, "version_minor": 0}

{"model_id": "ad569f70b6d64684aceff9a3e25e3b22", "version_major": 2, "version_minor": 0}

trainer = Trainer(
    model = model,
    args = args,

```



```
train_dataset = preprocessed_train_ds,  
eval_dataset = preprocessed_val_ds,  
compute_metrics = compute_metrics,  
)
```

Fine-tuning ViT

Fine-tuning the model by calling the `train()` method

```
trainer.train()  
  
<IPython.core.display.HTML object>  
  
TrainOutput(global_step=9156, training_loss=0.5893453882999845,  
metrics={'train_runtime': 4962.4647, 'train_samples_per_second':  
29.504, 'train_steps_per_second': 1.845, 'total_flos': 0.0,  
'train_loss': 0.5893453882999845, 'epoch': 6.0})
```

Evaluation on Test Set

The test score after fine-tuning

```
outputs = trainer.predict(preprocessed_test_ds)  
print(outputs.metrics)  
  
<IPython.core.display.HTML object>  
  
{'test_loss': 0.9034563899040222, 'test_accuracy': 0.6954583449428811,  
'test_runtime': 68.3209, 'test_samples_per_second': 52.532,  
'test_steps_per_second': 3.293}
```

The results on the test set as confusion matrix

```
y_true = outputs.label_ids  
y_pred = outputs.predictions.argmax(1)  
  
cm = confusion_matrix(y_true, y_pred)  
  
# plot with seaborn  
fig, ax = plt.subplots(figsize=(8,6))  
ax = sns.heatmap(cm, annot=True, fmt="d", linewidths=.5,  
xticklabels=string_labels, yticklabels=string_labels)  
plt.xlabel("Predicted")  
plt.ylabel("True")  
plt.show()
```

